

Intro - Slide 1

Introduction to Distributed Systems

Architecture

What it is:

Architecture is the **high-level structure** of a system.
It shows **what main parts exist** and **how they connect**.

Why it matters:

Architecture is chosen to achieve goals like:

- Fast performance
- Easy maintenance
- Good security
- Required functionality

Simple example:

Deciding that a system will have a **client** and a **server** that talk over the internet.

Software Architecture

What it is:

Software architecture is the **overall plan of a software system**.

It defines:

- Major components (client, server, database)
- How they communicate
- The style used (client–server, peer-to-peer, etc.)

Example:

A web app using **browser** → **web server** → **database**.

Software Design

What it is:

Software design is the **detailed design inside the architecture**.

It focuses on:

- Classes, functions, modules
- Data structures
- Implementation details

Relationship:

Architecture = big picture

Design = detailed implementation

Example:

Designing how the **login function** works inside a client-server system.

Client–Server Architecture

What it is:

A system where **clients request services** and **servers provide services**.

Client

What a client does:

- Starts the communication
- Sends requests
- Waits for a response
- Has a user interface (GUI)

Example:

Web browser, mobile app

Server

What a server does:

- Waits for client requests
- Processes requests
- Sends responses
- Handles many clients at once
- Usually has no GUI

Example:

Web server, database server

Peer-to-Peer (P2P) Computing

What it is:

Each computer (node) can act as **both client and server**.

Key ideas:

- No central server
- All nodes are equal
- Nodes connect directly

Used for:

File sharing, calls, media sharing

Examples:

Napster, KaZaA

Main problem:

Security is weak

Synchronous Communication

What it means:

The sender **waits** until the receiver replies.

Key idea:

Communication is **blocking**.

What happens:

1. Client sends request
2. Client stops and waits
3. Server replies
4. Client continues

Examples:

- Phone calls
 - Video meetings
 - RPC, RMI
 - Normal web request
-

Asynchronous Communication

What it means:

The sender **does not wait** for a reply.

Key idea:

Communication is **non-blocking**.

What happens:

1. Message is sent
2. Sender continues working
3. Receiver reads message later

Examples:

- Email
 - SMS
 - Message queues
 - Publish–Subscribe systems
-

Difference (Very Simple)

- **Synchronous:** Wait for reply
- **Asynchronous:** Don't wait

Request–Response Model

What it is:

A communication pattern where:

- Client sends a request
- Server sends a response

Important:

Each request is treated as **separate**.

Example:

HTTP communication

HTTP (Stateless Protocol)

What it means:

HTTP **does not remember previous requests**.

Key points:

- Uses request–response model
- No memory of past interactions
- Default port: **80**

Example:

Browser talking to a web server

How HTTP Keeps State

Since HTTP is stateless, extra methods are used:

- **Cookies:** Small data stored in browser
- **Server sessions:** Data stored on server
- **Hidden fields:** Data inside forms
- **URL parameters:** Data passed in URL

Purpose:

To remember users (login, carts, etc.)

Real-World Examples

- **Web browsing:** Synchronous
- **SMS:** Asynchronous

Intro to middleware - Slide 2

Introduction to Middleware

First: What is Middleware? (Big Picture)

Imagine this:

- At the **bottom**, you have the **Operating System** (Windows, Linux).
- At the **top**, you have **distributed applications** (web apps, mobile apps, services).
- These applications need to **talk to each other**, but:
 - They may use different languages
 - They may run on different machines
 - They may run on different OSes

👉 **Middleware is the middle layer that makes this communication possible and easier.**

What is Middleware (Simple Definition)

Middleware = software that sits between applications and the operating system.

It:

- Connects distributed applications
 - Hides low-level network details
 - Makes different systems work together
-

Why Middleware Exists (The Real Reason)

Without middleware:

- Developers must handle networking
- Handle message formats
- Handle failures
- Handle platform differences

With middleware:

- These are handled **for you**

👉 Middleware lets developers **focus on the application**, not the plumbing.

Types of Middleware

Middleware can be:

- **General-purpose** (used everywhere)
- **Special-purpose** (for specific needs)

Examples:

- ESB (Enterprise Service Bus)
 - Publish–Subscribe systems
-

Benefits of Middleware (Why We Use It)

Middleware works mainly because of **abstraction**.

It:

- Hides low-level details (network, OS)
 - Allows language independence (Java ↔ C++)
 - Allows platform independence (Windows ↔ Linux)
 - Encourages reuse of code and experience
 - Makes systems easier to change and extend
-

Drawbacks of Middleware

Nothing is free.

Middleware:

- Adds extra software layers → slower performance
 - Requires developers to learn middleware concepts
-

Middleware Architecture Properties

Middleware systems define:

1. Managed Entities

These are the things middleware manages:

- Objects
- Agents
- Components

2. Service Roles

Each entity has a role:

- Client
- Server
- Publisher
- Subscriber

3. Communication Style

Services communicate using:

- Synchronous communication
 - Asynchronous communication
-

Why Middleware is Important

Middleware exists because modern systems are:

- Distributed
 - Component-based
 - Built from old and new software
-

Reusing Legacy Software (Very Important)

Legacy software = old systems that still work but cannot be changed.

Problem:

- Old apps use their own interfaces
- They cannot directly connect to modern systems

Solution:

- Use middleware as a **common communication layer**
 - Build a **wrapper**
-

Wrapper (Easy Meaning)

A **wrapper**:

- Sits around the legacy app
- Translates its interface into middleware messages

👉 This allows old software to join new systems **without modification**.

Middleware as a Mediation System

Think of middleware as a **traffic controller**.

It can:

- Monitor performance
- Log errors
- Track usage
- Handle billing
- Do remote maintenance

- Install updates
-

Communication via Middleware

- All apps talk using a **common channel**
 - Usually done with **asynchronous messages**
 - Often implemented using a **message bus**
-

Publish–Subscribe Model

- Apps publish messages to a **topic**
- Other apps subscribe to that topic
- Publishers don't know receivers

👉 This reduces tight coupling.

Component-Based Architectures

Modern systems are built from **components**.

Each component:

- Has a clear responsibility
- Uses standard interfaces

Example:

- 3-tier architecture:
 - Presentation
 - Business logic
 - Database
-

Middleware's Role Here

Middleware provides:

- Runtime environment
- Transactions
- Security
- Communication

👉 Developers focus only on business logic.

Client Adaptation Using Proxies

Different clients = different needs.

Example:

- Mobile phone
- Desktop browser

Solution:

- Use a **proxy** between client and server
-

Proxy (Simple Meaning)

A **proxy**:

- Sits between client and server
 - Adjusts communication format
 - Optimizes data for the client device
-

Categories of Middleware (Intuition First)

Middleware differs based on:

1. Whether devices move
 2. Whether performance is predictable
-

Fixed & Predictable

- Devices don't move
 - Performance guaranteed
 - Used in real-time control systems
-

Fixed & Unpredictable

- Devices fixed
 - Performance varies
 - Internet systems
-

Mobile & Unpredictable

- Devices move
 - Wireless
 - Unstable performance
-

Middleware Structuring Principles

To manage complexity, middleware is structured.

Multi-Tier Architecture

Application is split into **tiers**.

Common tiers:

- Presentation
 - Business logic
 - Persistence (database)
-

Why Tiers Matter

- Each tier has a clear role
- Each tier can be scaled independently
- Improves maintainability and performance

Client Tier Variations

- UI only
 - UI + validation
 - Persistence split (ORM + DB)
-

Multi-Level Layered Architecture

Layers are stacked vertically.

Each layer:

- Has one responsibility
 - Uses only the layer below
 - Exposes an interface (like an API)
-

Key Rules

- Upper layer depends on lower layer
- Lower layer does not know upper layer
- Implementation is hidden

Example:

JVM, TCP/IP stack

Frameworks

A **framework** is a reusable program skeleton.

It:

- Provides structure
 - Controls flow
 - Allows customization through rules
-

How You Use a Framework

- Extend classes
- Implement interfaces
- Let the framework call your code (callbacks)

Examples:

Spring Boot, .NET

Distributed Objects

Objects that live on **different machines**.

They:

- Expose interfaces
 - Can be called remotely
 - Work across languages and platforms
-

How This Works

- Middleware handles communication
- Objects appear local but are remote

Examples:

RMI, CORBA

Remote Method Invocation (RMI)

RMI allows:

- Calling methods of remote objects
 - As if they were local
 - Middleware handles network details
-

Final Big Picture

Middleware is:

- The **glue** of distributed systems

- The **translator** between components
 - The **manager** of communication and services
-

Standard API vs Specific API (Middleware Context)

API (Application Programming Interface)

An API is a set of functions or rules that allows software components to communicate.

Standard API

A **standard API** is a commonly accepted interface used by many systems.

Features:

- Same rules everywhere
- Language and platform independent
- Improves interoperability and reuse

Examples:

HTTP, REST APIs, JDBC, POSIX

Role in middleware:

Middleware exposes standard APIs so different applications can communicate easily.

Specific API

A **specific API** is a custom or vendor-dependent interface designed for one system.

Features:

- Works only for a particular application
- Not portable
- Hard to replace or reuse

Examples:

Legacy system interfaces, vendor-specific APIs

Role in middleware:

Middleware uses **wrappers** to convert specific APIs into standard APIs.

Design and architecture patterns - Slide 3

Middleware and Architectural Patterns

(Story-style explanation + short notes)

1. What is a Pattern? (Start from real life)

In real life, we notice **things repeating**.

- Day → Night → Day
- Human body structure
- How houses are built
- How people react to danger (fight or flight)

👉 When something **keeps repeating and works well**, we call it a **pattern**.

Pattern

A **pattern** is:

- A **reusable solution**
- To a **problem that happens again and again**
- In a **predictable way**

Patterns exist in:

- Time (day, year)
- Structure (body, house)
- Logic (recursion)
- Behaviour (fight or flight)
- Emotions (anger, love)

2. Design Patterns (Zooming into Software)

In software, we also face **repeating problems**.

Example:

- How to create objects safely?
- How to connect incompatible components?
- How to notify many objects when something changes?

👉 **Design patterns** give **proven solutions** to these problems.

Design Pattern – Meaning

A **design pattern** is:

- A **general reusable solution**
- To a **common software design problem**
- NOT ready-made code
- A **template / idea**, not an algorithm

Important Clarifications (Exam gold)

- Design patterns solve **design problems**, not math problems
- Algorithms $\neq$ design patterns
- Not all patterns are design patterns

Design Pattern Categories

1. Creational Patterns

Concern: How objects are created

👉 Control object creation to avoid chaos.

Examples:

- Singleton – only one object
- Factory – object creation hidden
- Builder – complex object step by step

2. Structural Patterns

Concern: How objects are connected

👉 Focus on relationships and structure.

Examples:

- Adapter – make incompatible interfaces work
- Proxy – control access to object
- Facade – simplify complex systems

3. Behavioural Patterns

Concern: How objects communicate

👉 Focus on interaction and responsibility.

Examples:

- Observer – notify many automatically
 - Strategy – change algorithm dynamically
 - Mediator – reduce direct dependencies
-

3. Architectural Patterns (Bigger Picture)

Design patterns work at **class/object level**.

Architectural patterns work at **system level**.

Architectural Pattern – Meaning

An **architectural pattern**:

- Solves **high-level system design problems**
 - Defines the **structure of the entire system**
 - Is larger in scope than design patterns
-

Why Architectural Patterns Matter

They help solve:

- Performance problems
 - Availability problems
 - Scalability problems
 - Functional decomposition
-

Common Architectural Patterns

- Client–Server
- MVC
- 3-Tier
- Peer-to-Peer
- SOA
- Blackboard
- Publish–Subscribe

- Proxy / Broker
- Service Locator
- Microservices

4. Model–View–Controller (MVC)

What problem MVC solves

UI and business logic get mixed → hard to maintain.

MVC – Core Idea

👉 Separate responsibilities.

MVC Components

- **Model** – Data + business logic
 - **View** – User interface
 - **Controller** – Handles user actions and coordination
-

Why MVC is good

- UI can change without affecting logic
 - Logic can change without affecting UI
 - Easier maintenance and testing
-

MVC Communication (Triangle)

- View → Controller
- Controller → Model
- View reads from Model

MVC in Java EE (Jakarta EE)

Jakarta EE is a platform used to build large-scale Java web applications. It provides tools like Servlets, JSP, and APIs for database, security, and transactions.

MVC Architecture

MVC is a design pattern used to organize applications into three parts:

- **Model** → Handles data and business logic (Java classes, database)
- **View** → Displays data to the user (JSP, HTML)
- **Controller** → Handles user requests and controls flow (Servlets)

MVC in Java EE Mapping

- Model → Java Beans / Database (JPA)
- View → JSP
- Controller → Servlets

Flow

1. User sends request
2. Controller (Servlet) processes it
3. Gets data from Model
4. Sends data to View (JSP)
5. View displays response

Key Idea

MVC separates data, UI, and logic, making applications easier to manage, maintain, and scale.

5. 3-Tier Architecture

What it is

System split into:

1. Presentation tier
2. Business logic tier
3. Data tier

Difference: MVC vs 3-Tier

- **3-Tier:** Linear communication
- **MVC:** Triangular communication

👉 Both can coexist.

6. Peer-to-Peer Architecture

Idea

Connecting nodes in an adhoc manner without central control

- No central server
- Nodes connect directly
- Each node is both client and server

Used for

- File sharing
- Telephony

Examples:

Napster, KaZaA

7. Service-Oriented Architecture (SOA)

Why SOA exists

Large systems become:

- Hard to manage
- Hard to scale

SOA – Core Idea

Break system into **independent services**.

Each service:

- Has a clear interface
 - Does one business function
-

SOA Characteristics

- Distributed services
 - Service registry (to find services)
 - Orchestration (combine services)
 - Uses SOAP / REST
-

ESB (Enterprise Service Bus)

- Connects heterogeneous systems
 - Especially useful for legacy systems
 - Acts as orchestrator
-

SOA vs Microservices

- SOA = predecessor
 - Microservices = lighter, more decentralized
-

8. Blackboard Architectural Pattern

Idea

Inspired by AI.

How it works

- A **blackboard** = shared knowledge space
- Specialists read/write to blackboard
- A controller manages access

Why it's useful

- Components don't talk directly
 - Work independently
 - Solve complex problems gradually
-

Applications

- Speech recognition
 - Vehicle tracking
 - Protein analysis
-

9. Publish–Subscribe (Pub/Sub)

Core Idea

One publisher → many subscribers
No direct connection between them.

How it works

- Publisher sends messages to a **topic**
 - Subscribers listen to topics
 - Middleware sits in between
-

Why Pub/Sub is powerful

- Loose coupling
- Asynchronous
- Highly scalable

Where it's used

- Event notifications
- Live chat
- Stock markets
- IoT systems

Examples

RabbitMQ, ActiveMQ, JMS, Kafka

10. Proxy / Broker Pattern

Problem solved

Client should not talk directly to service implementation.

Proxy acts as the service and forward clients request to the service (middleman)

How it works

- Service exposes interface
- Proxy implements interface
- Client uses proxy
- Proxy forwards calls

Benefits

- Simpler client interface
- Hidden complexity
- Location transparency

Used in

RMI, CORBA

Proxy – Session Facade

Purpose: Hide complex backend and give clients a **simple interface**.

How it works:

1. **Client** calls **Session Bean (Facade / Proxy)**.
2. **Session Bean** talks to multiple **Entity Beans** (data objects / database).
3. **Session Bean** combines results and sends back to client.

Benefits:

- Hides internal complexity

- Client sees **simple methods**
 - Reduces direct interaction with database / entity beans
-

11. Service Locator Pattern

Problem

Client needs to find distributed services.

Solution

- Services register in a registry
- Client looks up service
- Location transparency achieved

Examples of Registries

JNDI, RMI Registry, UDDI

12. Cluster Architectural Patterns

Cluster Architecture:

Grouping multiple servers (nodes) to work together as one system.

Why clusters exist

- High availability
 - Load balancing
 - Fault tolerance
-

Cluster Types

- Single-tier cluster
- Multi-tier cluster

Middleware and distributed objects - Slide 4

Middleware and Distributed Objects

1. Start from what you already know: Client–Server

Normal Client–Server (No Middleware)

- Client knows:
 - Server IP
 - Port
 - Server protocol
- Client sends request
- Server replies

👉 Client **talks directly** to server.

Problem:

- Client must know networking details
 - Hard to change server
 - Hard to scale
-

2. What is RPC (Remote Procedure Call)?

Local Procedure Call (What you know)

`sum(a, b)`

- Function is in the same program
 - Direct call
-

Remote Procedure Call (RPC)

`sum(a, b) ← looks local`

But actually:

- Function runs on **another machine**
- Over the network

👉 **RPC makes a remote function look like a local one.**

3. How RPC Really Works (Calling functions remotely)

You **don't** call the server directly.

You call a **proxy**.

Client Side

- Client calls a **client proxy**
- Proxy packs data into bytes (marshalling)

Network

- Data sent over network

Server Side

- Server proxy unpacks data (**unmarshalling**)
- Calls real function
- Sends result back

👉 Middleware hides everything.

4. Objects: Local vs Remote

What is an Object? (Simple)

An object has:

- **State** → data
- **Behaviour** → methods
- **Interface** → what methods are visible

Local Object

- Object is in same machine
 - Method call is direct
-

Remote Object

- Object is on another machine
- Method call goes through middleware
- Looks local, works remote

👉 **Remote Object = Object + Network (hidden)**

5. Why Use Remote Objects?

Because we want:

- OOP benefits
 - In distributed systems
-

OOP Concepts Used

- **Abstraction** → user sees only interface
- **Encapsulation** → implementation hidden
- **Inheritance** → reuse code
- **Polymorphism** → same interface, different behavior

Middleware **extends OOP across machines**.

6. Why Remote Objects Are Perfect for Distributed Systems

1. Heterogeneity

- Different OS
- Different languages
- Different machines

👉 Remote objects hide differences.

2. Encapsulation

Client only knows:

- Interface

Does NOT care:

- Where object runs
 - How it is implemented
-

3. Dynamic Object Creation

- Objects can be created on remote machines
 - Middleware handles it
-

4. Inheritance & Polymorphism (Easy Example)

- Base class: `VideoStream`
- Child class: `ColorVideoStream`

Client accepts both → **polymorphism**

7. RMI (Remote Method Invocation) - **Calling methods (RPC OO version)**

What is RMI?

RMI is:

- RPC(calling functions)
- RMI (Calling remote methods in OOP)

Instead of calling functions:

- You call **methods on objects**

RMI in One Line

Calling a method on a remote object as if it were local.

8. RMI Architecture (Simple Flow)

1. Server creates remote object
 2. Server registers it in a registry
 3. Client looks up the object
 4. Client downloads a **proxy**
 5. Client calls method via proxy
 6. ORB(Object Request Broker) sends request to server
 7. Server executes method
 8. Result returned
-

9. ORB (Object Request Broker)

What is ORB?

ORB is:

- Middleware brain

- Handles communication
- Routes method calls

👉 Client never talks directly to server.

10. Proxy Pattern (Very Important)

What is a Proxy?

A proxy:

- Looks like the real object
 - Has the same interface
 - But no real logic
-

Why Proxy Exists

- Hides network details
 - Hides object location
 - Makes remote call look local
-

What Proxy Does Internally

1. **Pre-processing**
 - Convert parameters to bytes (marshalling)
2. **Send request**
3. **Receive reply**
4. **Post-processing**
 - Convert bytes back (unmarshalling)

11. Marshalling & Unmarshalling (Plain English)

Marshalling

- Convert object → byte stream
- To send over network

Unmarshalling

- Convert byte stream → object
-

Serialization vs Marshalling

- **Serialization:**
 - Converts an object into a **byte stream** to send over a network or save to disk
 - **Marshalling:**
 - Does everything serialization does, BUT also includes a **reference to the code** (class definition) if the receiver doesn't have it
-

12. Factory Pattern (Why Middleware Uses It)

Problem

Who creates remote objects?

Solution: Factory

- Factory creates objects
- Factory hides creation logic
- Factory may manage objects

Why Important in Middleware

- Remote object creation
 - Dynamic instances
 - Central control
-

13. Adapter Pattern (Wrapper)

Problem

Client expects one interface
Server provides another

Solution: Adapter

- Adapter converts interface

- Client stays unchanged
- Server stays unchanged

Example

REST → POX

Legacy API → Standard API

14. Interceptor Pattern

What is an Interceptor?

An interceptor:

- Sits in the call path
- Intercepts method calls
- Adds extra logic

What Interceptors Do

- Logging
- Security checks
- Load balancing
- Caching
- Redirect calls

Think of Interceptor as:

A security guard + traffic controller

15. Comparing Patterns (Very Easy)

Proxy vs Adapter

- Proxy → same interface
 - Adapter → different interface
-

Adapter vs Interceptor

- Adapter → changes interface
 - Interceptor → changes behavior
-

Proxy vs Interceptor

- Proxy → forwards calls
- Interceptor → adds logic

16. Mixing Patterns (Real Middleware)

Real middleware uses:

- Proxy
- Adapter
- Interceptor
together

Example:

- Proxy forwards call
- Interceptor adds security
- Adapter converts format

One-minute summary (memorize this)

- Middleware = middle helper
- RPC = remote function call
- Remote object = remote class instance
- Proxy = fake object you call
- ORB = message router
- Marshalling = data → bytes
- Standard API = common rules
- Specific API = private rules
- Adapter = translator
- Interceptor = guard

RMI Vs CORBA

1. What is Object-Oriented Middleware

👉 Middleware = software that sits **between client and server** and helps them communicate.

👉 Object-Oriented Middleware = everything is treated as **objects**.

So instead of:

Client → function call → server

You think:

Client → object.method() → remote object

2. Distributed Object Systems (CORE IDEA)

Definition (write like this in exam):

A distributed object system is a system where **objects are distributed across multiple machines** and communicate with each other.

Key idea:

- Objects can live on **different computers**
 - But you use them like **local objects**
-

Benefits

- ✓ Encapsulation → hide implementation
- ✓ Flexibility → object can be anywhere
- ✓ Reusability → wrap legacy systems
- ✓ Scalability → add more servers easily

👉 Simple line:

“Objects can be placed anywhere, but accessed uniformly.”

3. Remote Method Invocation (RMI)

Think:

👉 “Calling a method on another computer like it's local”

♦ RMI Design Goal

RMI allows a Java program to invoke methods on objects located on another machine.

◆ **RMI Architecture (VERY IMPORTANT)**

3 layers — memorize this:

1. Stub & Skeleton Layer

- Stub → client side (proxy)
- Skeleton → server side

👉 Job:

- Takes method call → converts to network call

2. Remote Reference Layer

- Manages and understands **object references**
- Knows **where object lives**

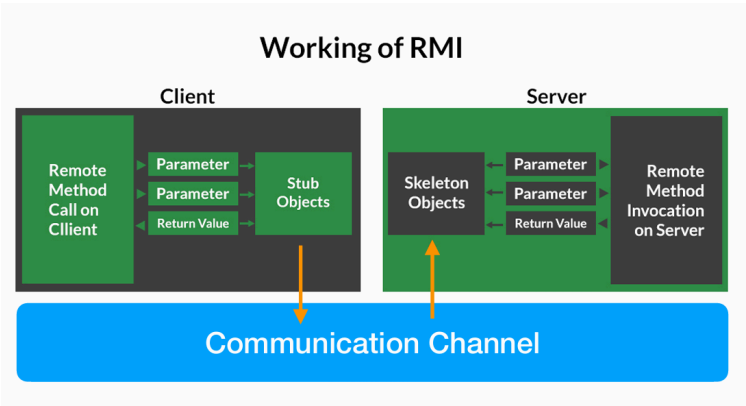
3. Transport Layer

- Uses **TCP/IP**
- Handles actual communication

Easy Flow

1. Client calls method
2. Stub intercepts
3. Request sent over network
4. Server executes method
5. Result returned

👉 That's it.



First Class	Second Class
-------------	--------------

Remote Naming Service (RMI)

- Used by clients to **locate remote objects/services** in a network
- Works like a **directory (phonebook)** of services

Server side	Client side
Real logic	Proxy / stub
Executes method	Forwards method

In RMI

- Uses **RMI Registry (rmiregistry)** as the naming service
- Runs on a known **host and port (default: 1099)**
- Server **registers** remote objects with a name
- Client **looks up** the name to get the service

Flow

1. Server registers service in registry
2. Client searches by name
3. Registry returns **proxy (stub)**
4. Client invokes remote methods through the proxy

What is CORBA

- System for distributed objects
- Works across different machines
- Works across different programming languages

👉 Goal: make different systems communicate

2. Who defines it? – Object Management Group (OMG)

- Group that defines CORBA standards
- Ensures all systems follow same rules

3. What is ORB?

- ORB = Object Request Broker
- Main part of CORBA
- Acts as **middleman**

👉 Does:

- Finds remote object
- Sends request
- Gets result back

👉 Gives: **location transparency** (client doesn't know where object is)

4. What is Interface Definition Language (IDL)

- Used to define interfaces
- Language independent
- Converted into Java, C++, etc.

5. Stub & Skeleton

- Stub → client side (proxy)
- Skeleton → server side
- Both generated from IDL

6. Process (Flow)

1. Define interface using IDL
2. Convert IDL → programming language
3. Generate stub & skeleton
4. Server implements object
5. Register object
6. Client looks up object
7. Client calls method
8. ORB handles communication

Feature	RMI	CORBA
Language Support	Java only	Multiple languages (Java, C++, Python...)
Middleware	Uses RMI runtime	Uses ORB
Interface Definition	Java interfaces	IDL (Interface Definition Language)
Client Proxy	Stub	Stub (generated from IDL)
Server Handler	Skeleton	Skeleton (generated from IDL)
Naming/Registry	RMI Registry (simple)	Naming Service + other CORBA services
Complexity	Simple	More complex, full enterprise support
Use Case	Java-only distributed apps	Heterogeneous distributed systems
Transparency	Java object-level	Location + language transparency

ESB Integration Patterns

1. Service-Oriented Architecture (SOA)

👉 SOA = build system using **independent services**

Each service does **one job**:

- Auth
- Balance check
- Transfer money

A Service-Oriented Architecture is a design where **loosely coupled services communicate over a network**.

SOA Components

1. Distributed Services

👉 Actual functionalities (Auth, Payment, etc.)

2. Registry (Name Service)

👉 Stores service locations

👉 Helps clients **find services**

3. Orchestrator

👉 Combines services to make a full workflow

Example:

Login → Check balance → Transfer → Confirmation

👉 Orchestrator = “controller”

4. ESB (comes here)

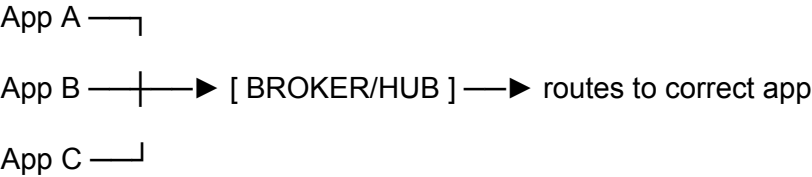
👉 Can act as the orchestrator

1. EAI (Enterprise Application Integration)

- Large companies have many separate systems (CRM, Supply Chain, BI, etc.)
 - These systems don't talk to each other by default
 - EAI connects them all using a middleware layer
 - Works **within a single enterprise** (internal only)
 - One integration layer → connects many apps (One to Many)
 - Examples of systems integrated: SCM, CRM, BI
-

2. EAI Model 1 — Broker (Hub & Spoke)

- One central engine called the **Broker/Hub**
- ALL messages must pass through the hub
- Hub handles: routing, transformation, data concurrency
- Example: JMS (Java Message Service)

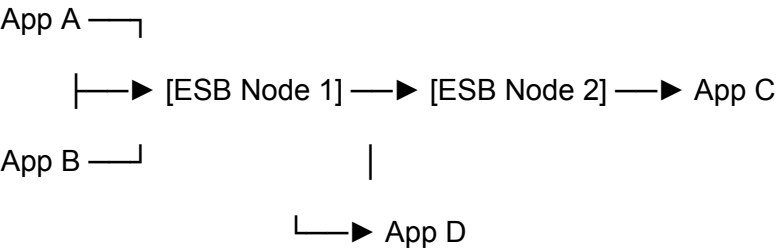


Why it's BAD:

- Hub crashes → everything stops ❌
- More apps = hub gets overloaded ❌
- Hard to scale ❌
- Single point of failure ❌

3. EAI Model 2 — ESB (Enterprise Service Bus)

- No single center — **distributed** across network
- Messages travel across a bus with multiple nodes
- Any node can fail — others keep working ✔️



Why it's GOOD:

- No single point of failure ✔️
- Scalable — add nodes easily ✔️
- Lightweight ✔️
- SOA friendly ✔️
- Incremental — add features step by step ✔️
- Built-in security, error handling, reliability ✔️

Examples: Apache, WSO2, TIBCO, WebLogic, WebSphere, JBOSS

4. Broker vs ESB

Feature	Broker	ESB
Structure	Centralized	Distributed
Single point failure	YES ❌	NO ✔️
Scalability	Hard	Easy
Lightweight	No	Yes

SOA Friendly	No	Yes
Example	JMS	Apache, WSO2

5. What ESB Does

- Monitors and controls message routing
- Resolves conflicts between services
- Manages service versions and deployment
- Uses backup services if one goes down
- Protocol conversion (HTTP ↔ TCP ↔ SOAP etc.)
- Data transformation (XML ↔ JSON ↔ CSV etc.)
- Message queuing — stores messages if receiver busy
- Message sequencing — delivers in correct order
- Service orchestration — coordinates multiple services
- Event handling and choreography
- Security enforcement

Popular ESB Tools:

Apache ESB → open source, widely used

WSO2 ESB → lightweight, SOA friendly

TIBCO → enterprise grade

WebLogic → Oracle's ESB

WebSphere → IBM's ESB

JBOSS → Red Hat's ESB

6. Integration Patterns

P1 — Message-Oriented Integration

- Apps don't talk directly — they send messages through ESB
- Sender doesn't wait — message delivered when receiver ready

App A → [MESSAGE] → ESB → [MESSAGE] → App B

P2 — Request-Reply

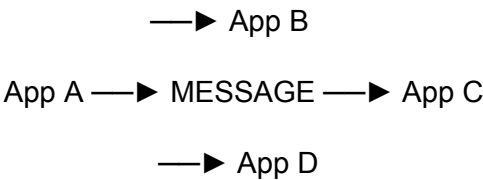
- App sends a request and waits for reply

App A → REQUEST → ESB → App B

App A ← REPLY ← ESB ← App B

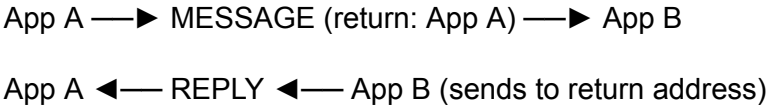
P3 — Multiple Consumers

- One message → received by many apps simultaneously



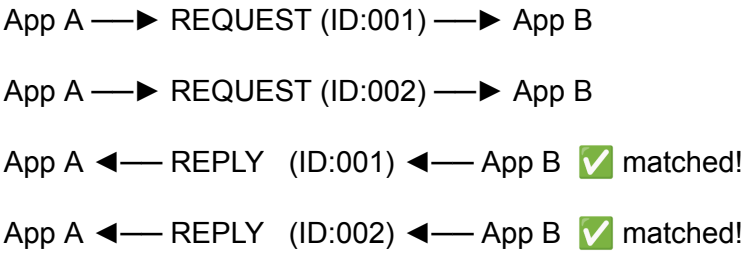
P4 — Return Address

- Sender includes own address in message
- Receiver knows where to send reply back



P5 — Correlation Identifier

- Each message gets a unique ID
- Replies matched back to correct request using ID



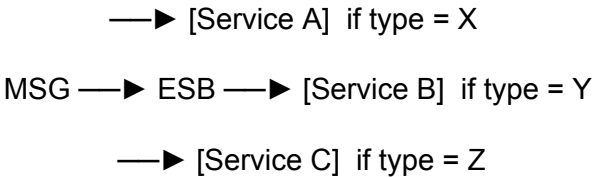
P6 — Pipes and Filters

- Message passes through a series of processing steps
- Each step does one small job



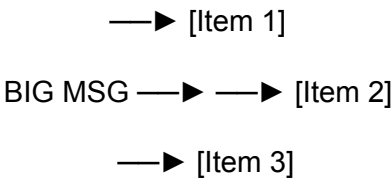
P7 — Content-Based Router

- ESB reads message content and routes to correct destination



P8 — Splitter

- One big message → split into many smaller messages



P9 — Splitter and Router

- Split the message THEN route each part to correct service
- ▶ [Item 1] —▶ Service A

BIG MSG —▶ —▶ [Item 2] —▶ Service B

—▶ [Item 3] —▶ Service C

P10 — Aggregator

- Opposite of splitter
- Many separate messages → combined into one

[Response 1] —┐

[Response 2] —┴—▶ AGGREGATOR —▶ ONE RESPONSE

[Response 3] —┐

P11 — Scatter-Gather

- Same message sent to many services simultaneously
- Waits for all responses → combines into one
- = Multiple Consumers + Aggregator

—▶ [Service 1] —┐

MSG —▶ —▶ [Service 2] —┴—▶ AGGREGATOR —▶ ONE RESPONSE

—▶ [Service 3] —┐

P12 — Composite Message

- Message has multiple parts
- Each part goes to a different service
- All responses gathered into one

—▶ [Part 1] —▶ Service A —┐

COMPOSITE —▶ —▶ [Part 2] —▶ Service B —┴—▶ ONE RESPONSE

—▶ [Part 3] —▶ Service C —┐

P13 — Multiple Service Providers

- Many services can do the same job
- ESB picks based on availability or load

—▶ [Provider 1] (if available)

MSG —▶ —▶ [Provider 2] (if P1 busy)

—▶ [Provider 3] (if P2 busy)

P14 — Multiple Specialised Providers

- Each provider does a different job
- ESB routes based on content/type

—▶ [XML Specialist] if format = XML

MSG —▶ —▶ [JSON Specialist] if format = JSON

—▶ [CSV Specialist] if format = CSV

P15 — Produce Single Response

- No matter how complex internally
- Client always gets ONE clean response

—▶ [Service 1] —┐

MSG —▶ —▶ [Service 2] —┐—▶ ESB —▶ ONE RESPONSE —▶ Client

—▶ [Service 3] —┐

7. ATM Exercise — ESB in Action

Services available:

1. Account Details (Card No) → HTTP / POX

2. Validate PIN → TCP / CSV

3. Add Balance → REST / JSON

4. Reduce Balance → SOAP

5. Check Account Balance → SOAP

6. Validate Account Number → REST / JSON

ATM sends all events via: HTTP + POX format

Event 1: LOGIN

- User enters: Card No + PIN
- ESB validates account (Service 6) + validates PIN (Service 2)
- Patterns: Composite Message, Protocol Conversion, Single Response

ATM —▶ ESB —▶ [Service 6: Validate Account No] REST/JSON

—▶ [Service 2: Validate PIN] TCP/CSV

—▶ ONE RESPONSE back to ATM 

Event 2: SHOW BALANCES

- ESB gets account details + balance
- Patterns: Composite Message, Aggregator, Single Response

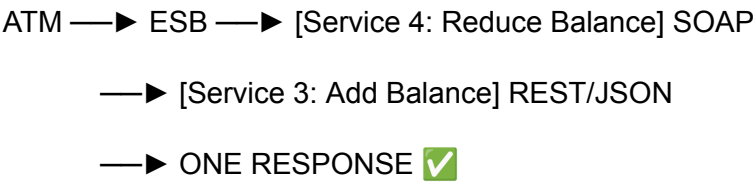
ATM —▶ ESB —▶ [Service 1: Account Details] POX/HTTP

—▶ [Service 5: Check Balance] SOAP

—▶ AGGREGATE —▶ ONE RESPONSE 

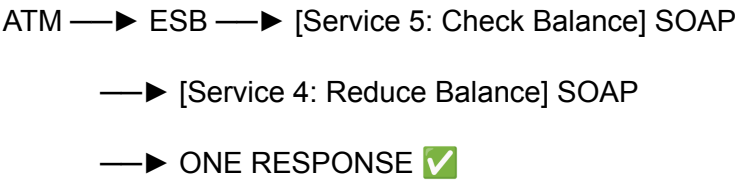
Event 3: TRANSFER FUNDS

- Reduce from account A, add to account B
- Patterns: Composite Message, Pipes & Filters, Single Response



Event 4: WITHDRAW CASH

- Check balance, then reduce balance
- Patterns: Pipes & Filters, Single Response



SUMMARY

TOPIC	KEY POINT
EAI	Connects enterprise apps via middleware
Broker Model	Central hub — single point of failure 
ESB Model	Distributed — scalable, reliable 
ESB Functions	Route, transform, convert, queue, secure
Integration Patterns	15 reusable templates for messaging problems
ATM Exercise	ESB orchestrates 6 services for 4 ATM events

Pattern Cheat Sheet:

Message-Oriented	→ apps talk via messages, not directly
Request-Reply	→ send request, wait for reply
Multiple Consumers	→ one message, many receivers
Return Address	→ include where to reply
Correlation ID	→ unique ID to match replies
Pipes & Filters	→ message through processing steps
Content-Based Router	→ route based on content
Splitter	→ one big message → many small ones
Splitter & Router	→ split then route each part
Aggregator	→ many messages → one combined
Scatter-Gather	→ send to many, collect all back
Composite Message	→ different parts to different services

Multiple Providers → same job, many providers

Specialised Providers → different jobs, different specialists

Single Response → always return one clean response

Microservices 1 & 2

Monolithic Applications

- One big application, packaged and deployed as a single unit.
- Example: **Java WAR file** deployed on Tomcat/Jetty.
- Easy to **develop, test, and deploy**.
- Scaling = run multiple copies behind a load balancer.

Drawbacks:

- Becomes very **large and complex**.
 - **Slow development** and long startup time.
 - Hard for **continuous deployment**.
 - **Scaling issues** if modules need different resources.
 - **Reliability issue** – one bug can crash the whole app.
 - Hard to **adopt new frameworks/languages**.
-

Microservices Applications

- Application is split into many **small, independent services**.
- Each service has its own function (e.g., order service, customer service).
- Services talk via **APIs (REST/RPC, messages)**.
- Each service can run in a **container (Docker)** or **VM**.
- Each service has **its own database schema** (not shared).

Benefits:

- Breaks complexity into **smaller, manageable services**.
- Each service can be **developed and deployed independently**.
- Enables **continuous deployment**.
- Services can be **scaled independently** (e.g., CPU-heavy vs memory-heavy).
- Teams can use **different technologies** per service (flexibility).

Drawbacks:

- More **complex** than monolith (distributed system).
- Must handle **communication failures** between services.
- **Database partitioning** makes transactions harder.

- **Testing** is more complex.
- **Deployment** is harder (many services, many instances).
- Requires tools like **Kubernetes + Docker** or a **PaaS** for automation.

Monolith vs Microservices (Quick Table)

Feature	Monolithic	Microservices
Structure	One big app	Many small services
Development	Simple at first	Complex setup
Deployment	Easy (copy one file)	Harder (many services)
Scaling	Whole app only	Each service independently
Reliability	One bug breaks all	Isolated failures
Flexibility	Hard to change tech	Easy to use mixed tech
Testing	Easy (end-to-end)	Harder (distributed testing)



👉 In short:

- **Monolith** = simple start, but hard to scale/manage when big.
- **Microservices** = more scalable and flexible, but more complex to build and run.

Scaling Cube

Quick Table

Axis	How it Works	Example
X	Clone whole app	Same app copied on 10 servers
Y	Split by function	Microservices: Auth, Orders, Payment
Z	Split by data/users	UserID ranges on different servers



👉 In short:

- **X** = copies
- **Y** = functions
- **Z** = data

Approach	Scaling Style	Why it matters	
Monolithic	Mostly X-axis	Just cloning the same big app, easy but limited	
Microservices	Y-axis (+ sometimes Z)	Break down into services, scale them independently	
Databases (often in microservices)	Z-axis	Split data across nodes for huge scale	

Three ways of handling client–service communication:

- 1. **Monolithic approach** → One big app. Client makes **one request**, server handles everything inside.
- 2. **Direct microservices call** → Client calls **each service separately** (many requests).
- 3. **API Gateway** → Client calls **one entry point**. Gateway routes requests to correct services.

Comparison Table		
Approach	Advantages	Disadvantages
Monolithic	Simple design One request only	Hard to scale Codebase huge
Direct Microservices	Clear separation No gateway needed	Many client requests Complex client logic
API Gateway	One entry point Client stays simple	Gateway adds extra layer If gateway fails, all fail

 How a Shopping App Works: From Monolith to Microservices

Imagine a **shopping app** with four main features: **Products, Orders, Payments, Notifications**.

1 Monolithic App

What it is: One big app where everything lives together.

How it works:

- Your mobile app requests `/product/123`.
- The server looks up product info, order info, payment info if needed—all in the same app.

Pros: 

- Simple to develop, test, and deploy.
- Can scale by copying the whole app behind a load balancer.

Cons: ❌

- Very large and complex, hard for one developer to understand.
- Slow to start and deploy.
- Hard to scale parts independently (e.g., image processing vs payments).
- One bug can crash the whole app.

2 Microservices App

What it is: Break the app into **small services**, each responsible for one feature.

Example:

- Product Service → manages product info
- Order Service → handles orders
- Payment Service → processes payments
- Notification Service → sends emails or push messages

Pros: ✅

- Easier to manage complexity.
- Teams can work independently on different services.
- Services can be scaled independently (CPU-heavy vs memory-heavy).
- Continuous deployment possible for each service.

Cons: ❌

- More complex system overall.
- Distributed system problems: failures, message delays.
- Testing is harder.

3 Inter-Process Communication (IPC)

What it is: How services talk to each other.

Monolith: Calls are just normal function calls inside the app.

Microservices: Calls happen over the network (HTTP, REST, or messaging queues).

Types of Communication:

- **Synchronous:** Service waits for response (**REST API**).
- **Asynchronous:** Service sends a message and continues (**RabbitMQ, Kafka**).

Example:

- Order Service asks Product Service for stock info → waits (sync) or gets response later (async).

API

Interface that lets software talk to each other. Instead of exposing database or logic directly, we expose **endpoints** (URLs).

REST API

Follows REST rules → use HTTP methods:

- **GET** → read
- **POST** → create
- **PUT** → update
- **DELETE** → remove

In Microservices

- Each service (e.g., *Student Service*) has its own business logic.
- API layer = wrapper that exposes this logic as endpoints.
- Client never calls functions directly; it calls APIs.

How it works (example)

- Endpoint: **GET** `http://localhost:3000/api/students/123`
- API layer → triggers `getStudent()` inside Student Service → returns result as JSON.

How we write APIs

- API layer usually in a **controller/route file**.
 - Service logic in **service folder**.
 - API calls → Controller → Service → Database.
-

4 Client-Service Interaction

- **One-to-One:** Mobile app asks Product Service → gets product details.
- **One-to-Many:** App places order → Order, Payment, and Notification Services all react.
- **Publish/Subscribe:** App publishes “Order Placed” → multiple services listen.

5 Handling Failures ⚠️

Microservices can fail. Strategies:

- **Timeouts:** Don't wait forever.
- **Circuit Breaker:** Stop calling failing service for a while.
- **Fallback:** Show cached data or defaults.

Example:

- Payment Service down → fallback shows “Try Cash on Delivery.”
-

6 Evolving APIs

Services change over time.

- Minor change: Add a **discount** field → old clients still work.
 - Major change: Version the API (`/api/v1/products` → `/api/v2/products`).
-

Service Discovery

Problem: Microservices run on different servers that can change → How do services find each other?

Solution: Service Registry = phone book for services.

Service Registration

- **Self-registration:** Service registers itself and sends heartbeats to keep registration alive.
 - **Pros:** Simple, no extra components needed.
 - **Cons:** Service coupled to registry; must implement registration logic.
- **Third-party registration:** Another system (**service registrar**) handles registering/unregistering services.
 - **Pros:** Services decoupled from registry; centralized management.
 - **Cons:** Extra component to manage; must be highly available.

Service Discovery Patterns

- **Client-side discovery:**
 - Client asks registry → finds service address → calls service directly.
 - **Pros:** Client can do smart, app-specific load balancing (e.g., consistent hashing).
 - **Cons:** Client coupled with registry; logic needed for each language/framework.
 - **Server-side discovery (with API Gateway or load balancer):**
 - Client connects to API Gateway → Gateway queries registry → finds service → routes request.
 - **Pros:** Clients don't worry about discovery; simpler client code.
 - **Cons:** Gateway must be highly available; extra moving part.
-

Example Flow

Mobile app → API Gateway → Order Service → Product Service → Response back

- Client never talks to registry directly.
 - API Gateway handles service lookup, routing, and can combine results from multiple services.
-

Examples of Service Registries

- **Netflix Eureka**: REST API, heartbeat every 30 sec.
 - **etcd**: Distributed key-value store, used in Kubernetes and Cloud Foundry.
 - **Consul**: Service discovery + health checks.
 - **Apache ZooKeeper**: Coordination service for distributed apps.
-

REST (R-E-S-T)

- **Representational State Transfer** → a way to communicate with services over **HTTP**.
 - Uses **URLs** (like `/orders/123`) + **methods** (GET, POST, PUT, DELETE).
 - Works with **APIs** → API = interface that exposes service functionality over REST.
 - Basically: **REST API = service you can talk to over the web using standard HTTP calls**.
-

Data in Microservices

1 How data is owned

- In microservices, **each service owns its own data**.
- Other services **cannot access it directly**. They must use the service's API.
- This keeps services **independent** and easy to change.

2 Polyglot Persistence

- Some services use **relational databases**, others use **NoSQL** or **graph databases**.
- Using different databases for different services is called **polyglot persistence**.
- Benefit: Services are **loosely coupled** and **perform better**.

3 Why polyglot persistence matters

- Each service can **choose the best database** for its needs.
- It helps scale services individually.
- But data is now **distributed**, so managing it is trickier.

4 Challenges of data in microservices

- **Transactions across services:** Hard to keep data consistent.
- **Querying multiple services:** Hard to get data from many services at once
- **Distributed transaction management:** Managing transactions across services is complicated, e.g., 2-Phase-Commit.
- **ACID properties:** Hard to guarantee full data reliability across all services.

💡 **Think of it like:**

Each service is a shop with its own storage. You can only get items by asking the shop. Shops can store items differently, but combining items from many shops is hard.

Event-Driven Architecture for Microservices

Why:

Helps microservices work together without tight connections.

How it works:

- Service publishes an event when something happens.
- Other services subscribe and react to events.
- Enables multi-service transactions.

Example (Order & Customer):

- Order Service → **Order Created** event.
- Customer Service → reserves credit → **Credit Reserved** event.
- Order Service → updates order status to **OPEN**.



Materialized Data Views (Pre-combined Data)

- Data from multiple services joined in advance.
- Updated automatically via events.
- Example: **Customer Order View**.



Benefits of Event-Driven Architecture

- Works across multiple services.
- Services stay independent.
- Queries are faster with materialized views.



Drawbacks of Event-Driven Architecture

- Programming is harder than normal transactions.
- Temporary data inconsistencies possible.
- Must handle duplicate events.
- Need compensating logic for failures.

Atomicity (Consistency of DB + Event)

- Database update + event publish must happen together.
 - Prevents system inconsistencies if service crashes.
 - Achieved via distributed transaction with message broker.
-

Choosing a Microservices Deployment Strategy

Why:

We have microservices. Now we need to decide **how to run them**. Different strategies give different trade-offs: cost, scaling, isolation, management.

Monolithic Application Deployment

- Single big app.
 - Run N servers, M copies per server.
 - Easy to deploy for small apps.
 - Hard to scale or manage for large apps.
-

Microservices Deployment Patterns

1. Multiple service instances per host

- Run many services on one server.
- Traditional way, simple.
- Each service uses a unique port.

2. Service instance per host

- One service per server.
- Can be:
 - **VM** → Full virtual machine, like EC2 instance.
 - **Container** → Lightweight OS-level virtualisation. Example: Docker.
- Provides isolation, easier to scale, manage resources.

3. Service instance per container

- Each service in its own container.
- Containers share OS but are isolated.
- Can limit CPU, memory, I/O.
- Example: Docker, Solaris Zones.

4. Serverless deployment

- No servers to manage.
- Example: AWS Lambda.
- Deploy as function ZIP.
- Automatically scales.
- Pay per execution.

Refactoring a Monolith into Microservices

Why:
Big monolith hard to scale and maintain. Microservices make it easier.

- How:**
- **Avoid Big Bang rewrite** → risky, likely fail.
 - **Incremental refactor** → gradually move features to microservices.

Strategies:

- 1. Stop Digging**
 - Don't add new code to monolith.
 - Put new features in microservices.
- 2. Split Frontend and Backend**
 - Separate presentation layer from business logic.
 - Frontend calls backend via API.
- 3. Extract Services**
 - Turn existing modules into independent microservices.
 - Shrinks monolith over time.

Aspect	VM	Container	
OS	Each VM has full OS	Shares host OS	
Size	Heavy (GBs)	Lightweight (MBs)	
Start time	Slow	Fast	
Isolation	Strong	Moderate but sufficient	
Microservices	Slower to scale	Perfect for scaling microservices	

Availiability

Fault, Error, Failure

Fault leads to Error, which can lead to Failure, but this is not guaranteed.

Definitions

Fault

- The cause of a problem
- Examples: programming bug, hardware malfunction, human mistake, natural disaster

Error

- Incorrect internal state of the system caused by a fault

Failure

- System behaves incorrectly or deviates from expected behavior
-

Important Points

- A fault may not cause an error. This is called a dormant fault.
 - An error may not cause a failure. This is called a latent error.
 - Latency is the time between the occurrence of a fault and the resulting failure. Long latency makes it harder to locate the fault.
 - A system fails whenever it is in an incorrect state.
-

Error Propagation

- A fault in one component can propagate and cause errors in other parts of the system, eventually leading to failure.

Classification of Failures

Fail-Stop

- Component stops working completely.
- Example: banking transfer cannot proceed.

Omission Failure

- Component fails to send or receive some messages but works normally otherwise.
- Example: payment gateway works intermittently.

Timing Failure

- Component works correctly but response or action is delayed.
- Example: delayed SMS OTP.

Byzantine Failure

- Component behaves unpredictably.
 - Example: conflicting or random outputs in a distributed system.
 - Hardest to handle, used for critical or hostile environments.
-

Availability vs Reliability

Availability

- System is ready to work when needed
- Measured by uptime

Reliability

- System works correctly over time
- Fewer failures = more reliable

Hindered by:

- 1. Failures (accidental)
- 1. Overload (too much demand)
- 1. Security attacks (deliberate)

Measuring Availability & Reliability

Key Metrics:

MTTF = Mean Time To Failure

- Average time until first failure

MTTR = Mean Time To Repair

- Average time to fix after failure

MTBF = Mean Time Between Failures

- **MTBF = MTTF + MTTR**

Availability Formula:

Availability = MTTF / (MTTF + MTTR) = MTTF / MTBF

"Number of 9s":

- 99.9% = Three 9s
 - 99.99% = Four 9s
 - 99.999% = Five 9s (5 minutes downtime per year)
-

Fault Tolerance with Redundancy

Fault tolerance is the ability of a system to keep working even when parts fail.

It uses extra resources to prevent failures from affecting the system.

Redundancy is the main method: extra information, processes, or copies help the system survive faults.

Three Types of Redundancy:

1. Information Redundancy

- Add extra data to detect/correct errors
- Examples: Parity, checksum, caching (Redis, proxy cache)

2. Temporal Redundancy

- Do same thing multiple times
- Examples: Triple Modular Redundancy, message queues

3. Spatial Redundancy

- Keep multiple copies of data
 - Examples: Load balancing, clustered database nodes
-

Fault Tolerance Techniques

1. Error Detection

- Find errors before they cause failure
- First step in fault tolerance

2. Error Compensation (Fault Masking)

- Hide errors using redundancy
- System works even with errors
- Example: Triple Modular Redundancy

3. Error Recovery

- Fix errors after detection
 - Two types:
 - Backward Recovery: Go back to previous good state
 - Forward Recovery: Fix current state to make it correct
-

Triple Modular Redundancy (TMR)

How it Works:

- Three identical components do same task
- Voter compares three outputs
- If one differs → discard it, use majority
- Voter also replicated (3 voters)

Assumption:

- Components fail independently
- Very unlikely all 3 fail at once

Result:

- System works even if 1 component fails

High-Availability Models

Who's active?

Active-Passive

- One active, one standby
- Standby only used when active fails
- Most hardware needed

Active-Active

- All nodes active
- Traffic shared or load balanced
- If one fails, others take over
- Needs same software on all nodes

How many nodes for each primary node?

N + 1

- N active nodes + 1 standby
- Standby can replace any failed node
- Good for multiple services **Example:** 3 servers + 1 backup. If any server fails, the backup takes over.

N + M

- N active nodes + M standbys
- More redundancy than N+1
- Trade-off: cost vs reliability

N to 1

- One powerful standby server
- Can run full system temporarily
- Original node restored later
Example: 3 servers fail one by one; standby handles each in turn, then original servers resume

N to N

- All nodes active
- Failed node's work redistributed
- No standby needed
- All nodes need extra capacity

Fault Recovery Strategies

Maintain State:

- Database: Persistent storage (survives crashes)
- Memory: Fast but lost on crash

Failover:

- Automatic switch to backup when primary fails

Load Balance + Failover:

- Distribute load + automatic failover
- Best availability

Tiered Redundancy

- Use multiple layers of redundancy for higher reliability.
- Example: servers + databases + network backups.

Containers

Containerised Architectures

1. Containerisation History

- 1970s: UNIX had **chroot**, to isolate file systems.
 - 1990s: FreeBSD made **jail** to divide computers into smaller systems.
 - 2005: Solaris OS made **containers** to run apps independently.
 - 2013: Docker made **containers easy, portable, and manageable**.
-

2. Virtualisation vs Containerisation

Virtualisation (VMs)

- Use a **hypervisor** to create virtual machines (VMs).
- Each VM has its **own OS**.
- VMs take **more memory and CPU**.
- Slow to start, heavy.

Containerisation

- Virtualises the **OS kernel**, not the full OS.
- Containers **share the host OS**.
- Lightweight, fast, and uses less memory.
- Easy to move between computers.

Key Difference:

- VM = own OS, heavy, slower.
 - Container = share OS, light, fast.
-

3. Docker

- Docker is a tool to **make and run containers**.
 - Holds apps, libraries, files, and dependencies in one **self-contained package**.
 - Works the same on any computer.
-

4. Kubernetes

- Manages **many containers** at once.
 - Automates **deployment, scaling, and maintenance**.
 - Makes sure containers keep running.
 - Starts new containers if one fails.
 - Other tools: Docker Swarm, Mesos.
-

5. Why Use Containers

- Traditional apps may work on one computer but fail on another.
- Containers **solve portability problems**.
- Write code once, run anywhere.

- Isolated apps do **not affect other programs**.

6. Benefits of Containers

- **Portable and reproducible** software.
- **Isolated and secure** – no interference with other apps.
- **Scalable** – easy to make more containers if needed.
- **Resource-efficient** – uses less memory and CPU than VMs.

7. VM vs Container

Feature	VM	Container
OS	Own OS	Share host OS
Start time	Slow	Fast
Memory/CPU	High	Low
Portability	Medium	High
Isolation	Strong	Strong

Docker & Kubernetes

Docker is a tool that packages your app into a container so it runs the same anywhere.

3 key things in Docker:

- **Dockerfile** — a text file you write with instructions ("use Python, copy my code, install these libraries")
- **Docker Image** — Docker reads your Dockerfile and builds a frozen, ready-to-run package that includes your actual code + everything it needs
- **Docker Container** — when you run the image, a live container starts. One image can make many containers.

Your code lives inside the image. That's the whole point — your code, your libraries, your environment, all in one box. No "works on my machine" problems.

Kubernetes comes in when you have lots of containers to manage.

- **Cluster** = a group of machines running your containers
- **Master node** = the brain — decides what runs where
- **Worker nodes** = the muscles — actually run your containers
- Kubernetes handles **auto-scaling**, **health checks**, and **load balancing** automatically. If a container crashes, Kubernetes restarts it for you.

Together in one line:

Docker *creates* containers. Kubernetes *manages* them at scale.

The 6-step flow:

1. Write your code + Dockerfile
2. Build a Docker Image
3. Push image to a registry (like Docker Hub)
4. Create a Kubernetes cluster
5. Deploy the image — Kubernetes runs it across nodes
6. Expose it publicly — your app is live!

MSA Patterns

Microservices Patterns – Plain English

The Problem

Monolith: one program. Everything runs together.

Microservices: many small programs. They talk over a network. Networks are slow. Networks fail.

You need patterns to handle this.

5 Patterns

1. API Gateway

One program sits in front of all your microservices. All requests go through it. It sends each request to the right microservice.

Result: Clients only know about the gateway, not every microservice.

2. Circuit Breaker

Stop calling a microservice that keeps failing. Track failures. After a set number of failures, stop all calls. Try again after some time.

States:

- Closed: calls go through
- Open: calls fail immediately
- Half-open: test one call to see if service is back

3. Saga

A business process spans multiple microservices. Each microservice has its own database. You cannot use a normal database transaction.

Break the process into small steps. Each step is a local transaction. If a step fails, run compensation steps to undo previous work.

Two coordination methods:

Choreography	Orchestration
No central controller. Services publish events. Other services listen.	One central service tells each service what to do and when.

Less coupling	Easier to track state
Hard to follow the flow	Creates a single point of failure

4. CQRS

Separate the write model from the read model. Commands change data. Queries read data.

Use different data structures or different databases for each.

5. Sidecar

Run a helper process next to your main service. It handles logging, monitoring, or network communication. Your main service stays simple.

Safety

Idempotency

Networks retry failed requests. Without protection, the same operation runs multiple times.

Client sends a unique key with each request. Server stores keys it has seen. When a key arrives:

- New key: process the request, store the result
- Seen key: return stored result without processing again

Three Safety Mechanisms

Mechanism	Purpose
Saga	Revert partial failures
Idempotency	Prevent duplicate execution
Dead Letter Queue	Store messages that keep failing for manual review

Code Patterns Inside a Service

Factory (Creational)

Create different objects based on input. Return email sender, SMS sender, or push sender depending on the channel parameter.

Adapter (Structural)

Wrap an external API with a consistent interface. If you change external providers, only the adapter changes. The rest of your code stays the same.

Strategy (Behavioral)

Swap algorithms at runtime. Use different pricing calculations for different days.

Summary Table

Pattern	Level	Problem Solved
API Gateway	Architecture	Clients know too many services
Circuit Breaker	Architecture	Repeated calls to failing services
Saga	Architecture	Transactions across databases
CQRS	Architecture	Read and write have different needs
Sidecar	Architecture	Every service needs logging/monitoring
Factory	Code	Different objects for different inputs
Adapter	Code	External APIs change
Strategy	Code	Different algorithms for different situations